

Fog-Assisted Vehicular Fog Computing

Module : Fog & Edge Computing

Présenté par :

Oussama BENYSSEF
Abdessamad EL FATHI
Mouad ASSARGUAL
Youssef LAGRAMEZ

Encadré par :

Pr. K. Ahed

Plan de la Présentation

- 1 Problématique
- 2 Objectif Principal
- 3 Méthodologie Adoptée
- 4 Paramètres de Simulation
- 5 Métriques Évaluées
- 6 Résultats Obtenus
- 7 Démonstration
- 8 Discussion
- 9 Conclusion et Perspectives

Plan

- 1 Problématique
- 2 Objectif Principal
- 3 Méthodologie Adoptée
- 4 Paramètres de Simulation
- 5 Métriques Évaluées
- 6 Résultats Obtenus
- 7 Démonstration
- 8 Discussion
- 9 Conclusion et Perspectives

Contexte : Réseaux Véhiculaires (VANET)

Définition :

- Réseau ad-hoc formé par véhicules communicants
- Communications V2V, V2I, V2N, V2P
- Mobilité élevée (50-150 km/h)
- Topologie dynamique et éphémère

Applications critiques :

- Évitement de collision (< 10 ms)
- Gestion intelligente du trafic
- Conduite autonome coopérative
- Services d'urgence et sécurité

Exigences temps réel

Sécurité routière : < 20 ms | Gestion trafic : < 100 ms | Fiabilité : 99.999%

Défis et Limites du Cloud Computing

Défis des VANET :

- **Congestion réseau** : Pics de trafic en heures de pointe
- **Latence élevée** : 100-250 ms via Cloud
- **Surcharge Cloud** : Centralisation des calculs
- Bande passante limitée (V2I)
- Mobilité rapide des nœuds

Limites du Cloud seul :

- × RTT trop élevé (~ 250 ms)
- × Point unique de défaillance
- × Coûts de bande passante élevés
- × Pas de conscience du contexte local
- × Inadapté au temps réel véhiculaire

Constat

Le Cloud seul **ne peut pas** répondre aux exigences de latence des applications véhiculaires temps réel. Il faut rapprocher le calcul des véhicules.

Plan

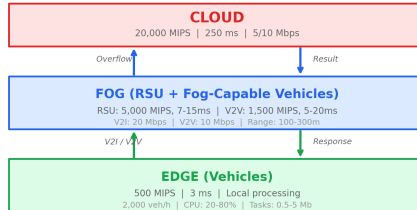
- 1 Problématique
- 2 Objectif Principal
- 3 Méthodologie Adoptée
- 4 Paramètres de Simulation
- 5 Métriques Évaluées
- 6 Résultats Obtenus
- 7 Démonstration
- 8 Discussion
- 9 Conclusion et Perspectives

Objectif du Projet

Objectif principal :

- 1 **Gérer la congestion** du trafic routier en temps réel via Fog Computing
- 2 **Optimiser le task offloading** dans un environnement véhiculaire multi-tiers
- 3 **Démontrer l'apport** du Fog/Edge Computing par rapport au Cloud seul

Architecture 3-tiers proposée :



Résultat clé

Réduction de latence jusqu'à **80%** grâce au Fog Computing véhiculaire.

Plan

- 1 Problématique
- 2 Objectif Principal
- 3 Méthodologie Adoptée**
- 4 Paramètres de Simulation
- 5 Métriques Évaluées
- 6 Résultats Obtenus
- 7 Démonstration
- 8 Discussion
- 9 Conclusion et Perspectives

3 scénarios comparatifs avec trafic synchronisé (2000 véh/h) :

Scenario	Fog Nodes	SCOOT	Cloud	Description
Edge + Cloud	0%	Disabled	Yes	Baseline (no fog)
Full Fog - Low	~10%	Active	Yes	Few fog nodes
Full Fog - High	~50%	Active	Yes	Many fog nodes

Principe de comparaison équitable :

- Même débit de trafic (2000 véh/h) et mêmes routes pour tous les scénarios
- Seule la **densité de fog nodes** varie entre les scénarios
- Les différences de latence observées sont **uniquement dues au fog**

Stratégie d'Offloading Multi-Critères

Modèle mathématique de décision :

Formule originale (Chen et al., 2016)

$$x^* = \arg \min_x \sum_{i=1}^N w_i \cdot f_i(x)$$

Optimisation convexe multi-utilisateur avec N fonctions objectif

Notre adaptation véhiculaire

$$d^* = \arg \min_d (T_d + \alpha L_d + \beta E_d)$$

Simplifié à 3 critères : temps, latence, énergie pour 4 destinations

Temps total d'offloading (Kumar et al., 2013 – repris tel quel) :

$$T_{\text{offload}} = T_{\text{upload}} + T_{\text{exec}} + T_{\text{download}} \quad (1)$$

$$T_{\text{upload}} = \frac{D_{\text{input}}}{BW_{\text{up}}}, \quad T_{\text{exec}} = \frac{I}{MIPS_d}, \quad T_{\text{download}} = \frac{D_{\text{output}}}{BW_{\text{down}}} \quad (2)$$

Réf. : Chen et al., IEEE/ACM Trans. Networking (2016) – Kumar et al., Mobile Networks & App. (2013)

Justification des Coefficients de Pondération

Pourquoi ces poids ? Approche inspirée de Chen et al. (2016), adaptée au contexte véhiculaire :

Critère	Poids	Justification
W_{latency}	0.35	Priorité maximale : les applications véhiculaires (sécurité, SCOOT) exigent <100 ms. C'est la contrainte la plus critique.
$W_{\text{task_size}}$	0.25	Les tâches lourdes (>2 Mb) bénéficient davantage de l'offloading vers des nœuds puissants (RSU, Cloud).
$W_{\text{congestion}}$	0.25	La congestion locale détermine si le RSU est surchargé (>8 véhicules) et dicte le mécanisme d'overflow.
$W_{\text{fog_avail}}$	0.15	Critère secondaire : la disponibilité fog est vérifiée seulement après les critères principaux.

Validation expérimentale

$\sum W_i = 1.0$ Les coefficients sont validés par la simulation : la latence étant le facteur dominant, son poids plus élevé (0.35) favorise les destinations à faible latence (RSU, Fog) plutôt que le Cloud.

Modèles de Latence Dynamique

1. Latence V2V (Fog) :

Concept original (Hou et al., 2016)

"La latence V2V décroît quand la densité de fog nodes augmente car la probabilité de trouver un relai proche augmente." *Modèle probabiliste, pas de formule fermée.*

Notre modélisation linéaire

$$L_{V2V} = \max(5, 20 - \rho_{\text{fog}} \times 30)$$

Approximation linéaire : 20 ms ($\rho=0$) $\rightarrow$ 5 ms ($\rho=0.5$)

2. Latence RSU :

Concept original (Yousefpour et al., 2019)

"La latence RSU dépend de la charge et de la distance au RSU. La file d'attente au RSU augmente avec le nombre de requêtes."

Notre modélisation

$$L_{RSU} = \max(7, 15 - \rho \times 8) + \frac{d}{300} \times 5$$

Deux composantes : charge (ρ) + distance ($d/300\text{m}$)

Réf. : Hou et al., IEEE Trans. Vehicular Technology (2016) – Yousefpour et al., J. Systems Architecture (2019)

Mécanisme de Capacité RSU et Overflow

Règle clé : Chaque RSU a une capacité limitée (~ 8 véhicules simultanés)

Mécanisme d'overflow en cascade

- 1 RSU pas surchargé $\rightarrow$ **RSU** (7-15 ms)
- 2 RSU surchargé + fog nearby $\rightarrow$ **FOG_VEHICLE** (5-20 ms)
- 3 RSU surchargé + pas de fog $\rightarrow$ **CLOUD** (250 ms) $\leftarrow$ *pénalité massive*

C'est ce mécanisme qui crée la différence entre les scénarios :

- **Edge+Cloud** (0% fog) : pas d'étape 2 $\Rightarrow$ overflow direct vers Cloud (250 ms)
- **High Fog** (50% fog) : l'étape 2 absorbe la majorité $\Rightarrow$ Cloud réduit à 8%

Code : Décision d'Offloading (iFogSim/Java)

VehicularFogSimulation.java – calculateOffloadingDecision()

```
// RSU surcharge si >8 vehicules dans la zone
boolean rsuOverloaded = congestionNorm > 0.32;
double fogDensityRatio = (totalVehicles > 0) ?
    (double) fogNodes / totalVehicles : 0.0;

// Latences dynamiques
double dynamicV2VLatency = Math.max(5.0, 20.0 - fogDensityRatio * 30.0);
double dynamicRSULatency = Math.max(7.0, 15.0 - fogDensityRatio * 8.0);
if (rsuOverloaded) {
    dynamicRSULatency *= (1.0 + congestionNorm * 2.0); // Penalite surcharge
}
double L_cloud = LATENCY_CLOUD * 2 + 150.0; // ~250ms

// Decision basee sur capacite RSU
if (cpuLoad < CPU_THRESHOLD_LOW && taskSizeNorm < 0.3)
    bestDest = "LOCAL"; // Petite tache -> local (3ms)
else if (isInRSUZone && !rsuOverloaded)
    bestDest = "RSU"; // RSU dispo -> RSU (7-15ms)
else if (hasFogVehicleNearby)
    bestDest = "FOG_VEHICLE"; // Fog nearby -> V2V (5-20ms)
else
    bestDest = "CLOUD"; // Dernier recours -> Cloud (250ms)
```

Outil	Rôle dans la simulation
SUMO	Simulateur microscopique de trafic routier. Gère les véhicules, feux et routes via l'API TraCI.
iFogSim	Simulateur Fog Computing (Java). Calcule les décisions d'offloading et les métriques.
Python/Flask	Orchestrateur central + Dashboard temps réel (Socket.IO).
SCOOT	Algorithme adaptatif de contrôle des feux (Split, Cycle, Offset).
TraCI	API Python pour contrôler SUMO en temps réel.

Communication inter-composants :

SUMO TraCI (Python) Orchestrateur (server.py) Socket TCP 5555 iFogSim (Java)

Algorithme SCOOT Adaptatif

SCOOT (Split Cycle Offset Optimization Technique) – Contrôle adaptatif des feux :

Principe :

- Mesure des files d'attente en temps réel
- Ajustement dynamique des phases de feux
- Optimisation par zone RSU
- 3 types de décisions : **split**, **cycle**, **offset**

SCOOTController.java

```
// Calcul du split optimal
double nsRatio = nsQueue /
    (nsQueue + ewQueue + 1);
int nsGreen = (int)(cycleLength
    * nsRatio);
int ewGreen = cycleLength - nsGreen;

// Seuils d'intervention
if (nsQueue > ewQueue * 1.5) {
    decision = "BASCULER vers N-S";
}
```

Formule du Performance Index (PI) :

$$PI = \frac{Q_{\max}}{Q_{\text{capacity}}} \quad \text{où } Q_{\max} = \max(Q_{NS}, Q_{EW}) \quad (3)$$

Code : Configuration SUMO et TraCI

routes_low_fog.rou.xml (10% fog)

```
<!-- Types de vehicules -->
<vType id="car" maxSpeed="50"
  guiShape="passenger"
  color="0,0,255"/>
<vType id="fog_capable" maxSpeed="55"
  guiShape="passenger/sedan"
  color="0,255,0"/>

<!-- Route WE: 500 veh/h -->
<!-- 90% car, 10% fog -->
<flow id="we_car" type="car"
  route="route_we"
  begin="0" end="300"
  vehsPerHour="450"/>
<flow id="we_fog" type="fog_capable"
  route="route_we"
  begin="0" end="300"
  vehsPerHour="50"/>
```

server.py – TraCI (Python)

```
import traci

# Connexion SUMO via TraCI
traci.start(["sumo-gui", "-c",
  sumo_config_path])

# Boucle de simulation
while traci.simulation.getTime() < 300:
  traci.simulationStep()

  # Recuperer les vehicules
  for veh_id in traci.vehicle\
    .getIDList():
    x, y = traci.vehicle\
      .getPosition(veh_id)
    speed = traci.vehicle\
      .getSpeed(veh_id)
    vtype = traci.vehicle\
      .getTypeID(veh_id)

  # Changer les feux (SCOOT)
  traci.trafficlight.setPhase(
    tl_id, new_phase)
```

Plan

- 1 Problématique
- 2 Objectif Principal
- 3 Méthodologie Adoptée
- 4 Paramètres de Simulation**
- 5 Métriques Évaluées
- 6 Résultats Obtenus
- 7 Démonstration
- 8 Discussion
- 9 Conclusion et Perspectives

Paramètres de Simulation

Network Infrastructure

Parameter	Value
Simulation duration	300 s
Number of RSUs	4 (at intersections)
RSU range	300 m
RSU capacity	~8 simultaneous veh.
V2V range	100 m
Vehicle throughput	2000 veh/h
Number of roads	5

Computing Capacities

Level	MIPS
Local vehicle (Edge)	500
Fog-capable vehicle	1,500
RSU	5,000
Cloud	20,000

Generated Tasks

Parameter	Value
Input size (D_input)	0.5 - 5.0 Mb
Output size (D_output)	20% of input
Instructions (I)	50 - 500 MI
CPU load	20 - 80%

Bandwidth and Latency

Link	Throughput	Latency
V2V (fog)	10 Mbps	5-20 ms
V2I (RSU)	20 Mbps	7-15 ms
Cloud up	5 Mbps	250 ms
Cloud down	10 Mbps	250 ms

Traffic Scenario Details

Parameter	Edge+Cloud	Low Fog	High Fog
Total throughput	2000 veh/h	2000 veh/h	2000 veh/h
Standard vehicles	100%	90%	50%
Fog-capable vehicles	0%	10%	50%
Fog enabled	No	Yes	Yes
SCOOT enabled	No	Yes	Yes
Route file	routes.rou.xml	routes_low_fog	routes_high_fog

Coûts énergétiques : (adapté de Miettinen & Nurminen, 2010)

- Calcul local : $E_{\text{compute}} = I \times 0.5 \text{ J/Mi}$ – processeur embarqué, forte consommation
- Calcul fog : $E_{\text{compute}} = I \times 0.1 \text{ J/Mi}$ – 5× plus efficace (ressources mutualisées)
- Transmission : $E_{\text{transmit}} = (D_{\text{in}} + D_{\text{out}}) \times 0.01 \text{ J/Mb}$

Plan

- 1 Problématique
- 2 Objectif Principal
- 3 Méthodologie Adoptée
- 4 Paramètres de Simulation
- 5 Métriques Évaluées**
- 6 Résultats Obtenus
- 7 Démonstration
- 8 Discussion
- 9 Conclusion et Perspectives

Evaluated Metrics

Metric	Description
Average latency	Weighted average task processing time
Processing distribution	Edge / Fog / Cloud breakdown (%)
Active Fog Nodes	Number of detected fog-capable vehicles
Adjusted traffic lights	SCOOT decisions applied to traffic lights
Rerouted vehicles	Reroutings following congestion detection
Max congestion	Maximum saturation rate per RSU zone
Energy per task	Weighted energy consumption per task (mj)

Formules des Métriques : Originales vs Adaptées

1. Énergie de calcul

Miettinen & Nurminen (2010)

$$E = P_{\text{cpu}} \cdot T_{\text{exec}} \quad P_{\text{cpu}} : \text{puissance selon l'appareil (0.5–3 W mobile)}$$

Notre simplification

$$E = I \cdot c_{\text{tier}} \quad q_{\text{ocal}} = 0.5 ; c_{\text{fog}} = 0.1 \text{ J/MI (Ratio 5 :1)}$$

2. Congestion par zone

Webster (1958) – Degré de saturation

$$x = \frac{q}{s \cdot (g/C)} \quad q : \text{débit}, s : \text{saturation}, g : \text{vert effectif}, C : \text{cycle}$$

Notre normalisation

$$C_{\text{zone}} = \min\left(1, \frac{N}{25}\right) \quad \text{Seuil 25 : capacité zone RSU (300 m, 4 RSU)}$$

3. Latence moyenne (Deng et al., 2016 – moyenne pondérée)

$$L_{\text{avg}} = \frac{L_{\text{edge}} P_{\text{edge}} + L_{\text{fog}} P_{\text{fog}} + L_{\text{cloud}} P_{\text{cloud}}}{100}$$

Réf. : Miettinen & Nurminen (2010) – Webster (1958) – Deng et al. (2016)

Plan

- 1 Problématique
- 2 Objectif Principal
- 3 Méthodologie Adoptée
- 4 Paramètres de Simulation
- 5 Métriques Évaluées
- 6 Résultats Obtenus**
- 7 Démonstration
- 8 Discussion
- 9 Conclusion et Perspectives

Résultats : Tableau Comparatif

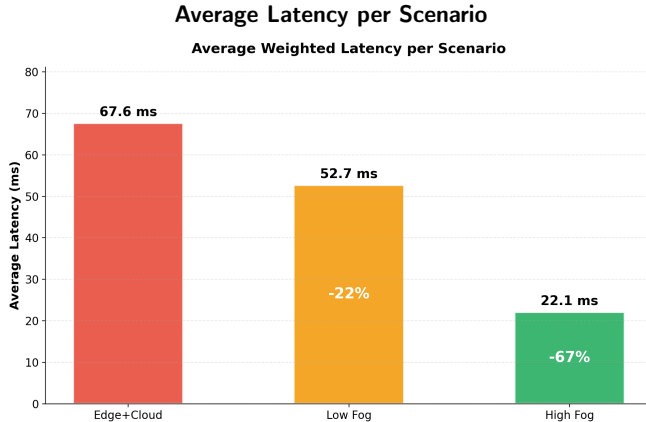
Comparative Results

Metric	Edge+Cloud	Low Fog (10%)	High Fog (50%)
Max vehicles	41	41	47
Active Fog Nodes	0	5 (12%)	22 (51%)
Adjusted lights (SCOOT)	0	107	146
Rerouted vehicles	0	1,959	4,296
Max congestion	100%	93%	100%
Edge latency	5 ms	5 ms	5 ms
Fog latency	--	22 ms	22 ms
Cloud latency	130 ms	130 ms	130 ms
Average latency	~68 ms	~53 ms (-22%)	~22 ms (-67%)
Energy per task	342 mJ	148 mJ (-57%)	139 mJ (-59%)

Observation clé

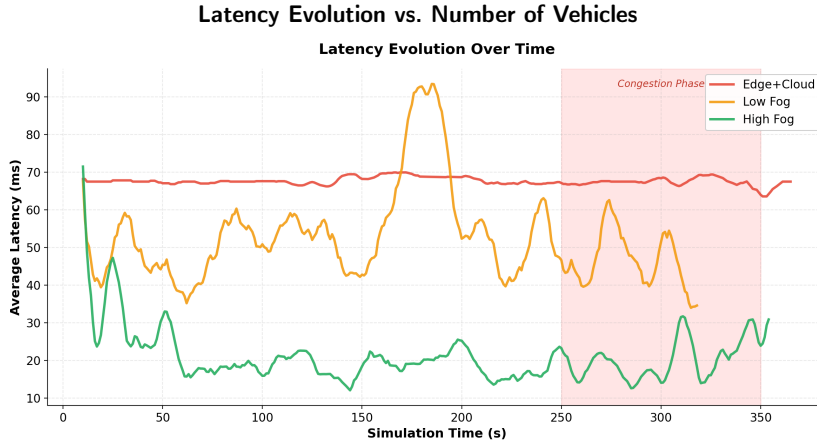
Le Fog Computing réduit la latence de **22%** (Low Fog) à **67%** (High Fog) par rapport au scénario Edge+Cloud.

Résultats : Latence Moyenne par Scénario



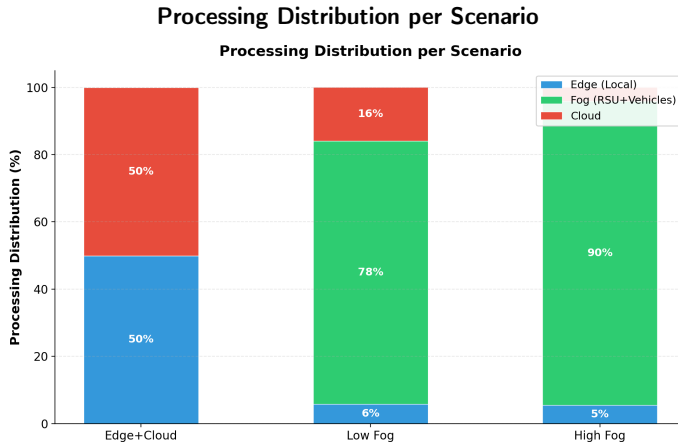
Réduction : Low Fog : -22% (68→53 ms) High Fog : -67% (68→22 ms)

Résultats : Évolution de la Latence



Observation : Plus le nombre de véhicules augmente, plus l'écart de latence se creuse entre les scénarios. Le fog maintient une latence stable.

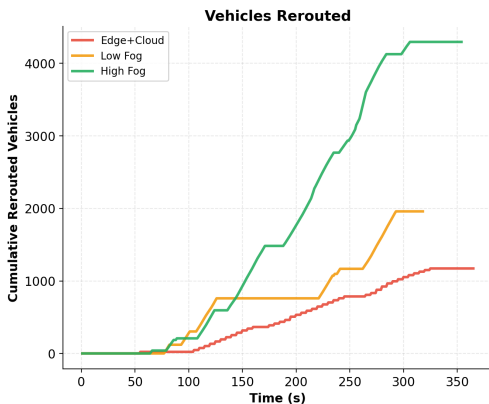
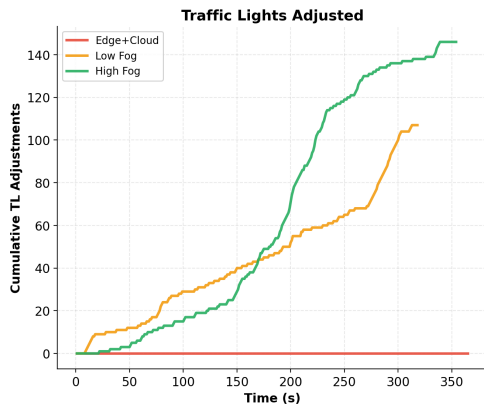
Résultats : Distribution du Traitement



Interprétation : Avec 50% de fog nodes, **90%** des tâches sont traitées par le fog, réduisant l'usage du cloud de 50% à 4%.

Résultats : Actions Fog (Feux Ajustés et Reroutages)

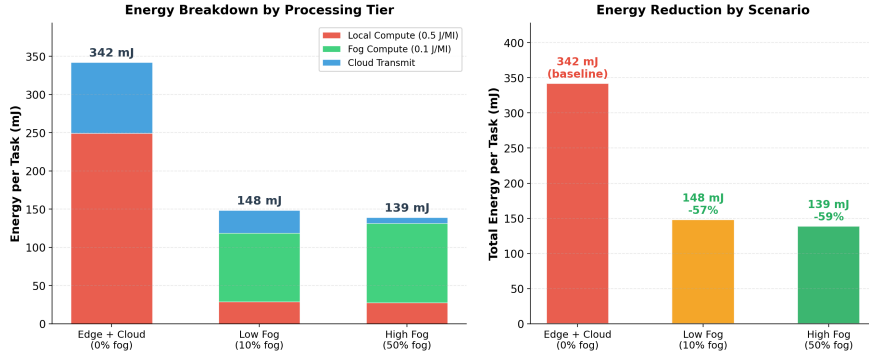
Fog Actions : Adjusted Lights & Rerouted Vehicles



Impact du Fog : Plus de fog nodes $\Rightarrow$ plus de décisions locales, plus de réactivité. SCOOT ajuste **146 feux** en High Fog (vs 0 en Edge+Cloud) et reroute **4296 véhicules**.

Résultats : Consommation Énergétique

Energy Consumption Comparison per Scenario



Observations :

- Le fog est **5× plus efficace** en énergie que le calcul local (0.1 vs 0.5 J/MI)
- **High Fog** : réduction de **59%** de l'énergie par tâche (342→139 mJ)
- Le coût énergétique diminue car le fog absorbe les tâches lourdes

Résultats : Exemples de Décisions SCOOT en Temps Réel

Décisions SCOOT observées (High Fog, T=60-120s) :

```
[SCOOT SPLIT] J2: EW=32s, NS=18s (PI=0.00, NS queue=0m, EW queue=7m)
[EXEC FOG->SUMO] J1: BASCULE vers N-S - NS:5 EW:0
[EXEC FOG->SUMO] RSU_J1: 25 vehicules REROUTES
[EXEC FOG->SUMO] J2: Vert E-W etendu a 40s
[EXEC FOG->SUMO] J3: Vert N-S etendu a 40s
```

Interprétation :

- **PI** (Performance Index) : 0 = fluide, 1 = bloqué
- **Split** : Répartition du temps de vert (ex : NS=18s, EW=32s)
- **Reroutage** : 25 véhicules redirigés pour éviter la congestion
- **Extension** : Phase verte étendue si file d'attente importante
- Toutes les décisions prises en < **50 ms** grâce au traitement fog local

Plan

- 1 Problématique
- 2 Objectif Principal
- 3 Méthodologie Adoptée
- 4 Paramètres de Simulation
- 5 Métriques Évaluées
- 6 Résultats Obtenus
- 7 Démonstration**
- 8 Discussion
- 9 Conclusion et Perspectives

Lancement des 3 scénarios de simulation



Scénario 1

Edge + Cloud

(0% fog)

Référence

SCOOT désactivé

Cloud seul (~130 ms)



Scénario 2

Low Fog

(10% fog nodes)

Fog limité

SCOOT actif

Latence ~53 ms



Scénario 3

High Fog

(50% fog nodes)

Fog dense

SCOOT actif

Latence ~22 ms

À observer

Dashboard temps réel • Ajustements SCOOT • Reroutages • Latence • Distribution du traitement

Plan

- 1 Problématique
- 2 Objectif Principal
- 3 Méthodologie Adoptée
- 4 Paramètres de Simulation
- 5 Métriques Évaluées
- 6 Résultats Obtenus
- 7 Démonstration
- 8 Discussion**
- 9 Conclusion et Perspectives

Observations principales :

- ❶ **Réduction de latence significative** : jusqu'à 67% avec High Fog
 - Le fog absorbe l'overflow RSU (5-22 ms vs 130 ms cloud)
 - Plus de fog nodes = meilleure couverture V2V = moins de cloud
- ❷ **SCOOT beaucoup plus efficace avec le fog**
 - High Fog : 146 ajustements vs Edge+Cloud : 0 ajustement
 - La latence réduite permet des décisions plus fréquentes
- ❸ **Gestion proactive de la congestion**
 - 4296 reroutages en High Fog vs 1172 en Edge+Cloud
 - Le fog permet des décisions proactives de reroutage grâce à la faible latence
- ❹ **Résultat clé** : High Fog gère la congestion **plus efficacement**
 - Le SCOOT + reroutage anticipe et distribue le trafic

Avantages et Limites du Fog Véhiculaire

Avantages démontrés :

- ✓ Latence réduite : 68 ms $\rightarrow$ 22 ms (-67%)
- ✓ Décisions temps réel (< 50 ms)
- ✓ Pas de point unique de défaillance
- ✓ Économie de bande passante cloud
- ✓ Conscience du contexte local
- ✓ Scalabilité proportionnelle au trafic

Limites identifiées :

- ✗ Performance dépend de la densité fog
- ✗ Mobilité = voisins fog instables
- ✗ Hétérogénéité des ressources véhiculaires
- ✗ Nécessite infrastructure RSU
- ✗ Sécurité et authentification complexes
- ✗ Pas de fog en zones rurales peu denses

Seuil d'efficacité

À partir de $\sim 10\%$ de fog nodes, les bénéfices sont significatifs (-22% latence, -57% énergie). Le passage à 50% apporte un gain supplémentaire (-67% latence, -59% énergie).

Plan

- 1 Problématique
- 2 Objectif Principal
- 3 Méthodologie Adoptée
- 4 Paramètres de Simulation
- 5 Métriques Évaluées
- 6 Résultats Obtenus
- 7 Démonstration
- 8 Discussion
- 9 Conclusion et Perspectives

Ce que nous avons démontré :

- ① Le **Fog Computing** réduit la latence de **67%** ($68 \rightarrow 22$ ms) et l'énergie de **59%** ($342 \rightarrow 139$ mJ)
- ② L'algorithme **SCOOT** optimise les feux en temps réel (146 ajustements, 4296 reroutages)
- ③ Le mécanisme de **capacité RSU** avec overflow vers fog/cloud crée une différenciation réaliste
- ④ Plus de **fog nodes** = meilleure gestion du trafic et congestion réduite
- ⑤ L'architecture **Edge-Fog-Cloud** est efficace et scalable pour les VANET

Message clé

Le Fog Computing Véhiculaire n'est pas une alternative au Cloud, mais un **complément essentiel** qui rapproche le calcul des véhicules. Il permet des décisions en **temps réel** là où le Cloud échoue.

Améliorations futures :

- **Machine Learning** : Prédiction de congestion par modèles ML (LSTM, GNN)
- **5G/6G C-V2X** : Intégration pour latence < 1 ms
- **Federated Learning** : Apprentissage distribué coopératif sur véhicules
- **Sécurité** : PKI véhiculaire (IEEE 1609.2)
- **Platooning** : Application aux convois de camions autonomes
- **Multi-hop V2V** : Extension de la portée fog par relais

Évolution technologique :

Parameter	DSRC (current)	5G C-V2X	6G (future)
Latency	20-50 ms	1-10 ms	< 1 ms
Throughput	27 Mbps	1 Gbps	1 Tbps
Range	300 m	1 km	10 km

Références (1/2)



Bonomi, F., Milito, R., Zhu, J., Addepalli, S. (2012). "Fog Computing and Its Role in the Internet of Things." *Proc. MCC Workshop on Mobile Cloud Computing*, ACM. <https://doi.org/10.1145/2342509.2342513>



Vaquero, L.M., Rodero-Merino, L. (2014). "Finding Your Way in the Fog : Towards a Comprehensive Definition of Fog Computing." *ACM SIGCOMM Computer Communication Review*, 44(5). <https://doi.org/10.1145/2677046.2677052>



Hou, X., Li, Y., Chen, M., Wu, D., Jin, D., Chen, S. (2016). "Vehicular Fog Computing : A Viewpoint of Vehicles as the Infrastructures." *IEEE Trans. on Vehicular Technology*, 65(6). <https://doi.org/10.1109/TVT.2016.2532863>



Ning, Z., Dong, P., Wang, X., Rodrigues, J. et al. (2019). "Deep Reinforcement Learning for Vehicular Edge Computing." *IEEE Trans. on Cognitive Communications and Networking*, 5(4). <https://doi.org/10.1109/TCCN.2019.2930521>



Mach, P., Becvar, Z. (2017). "Mobile Edge Computing : A Survey on Architecture and Computation Offloading." *IEEE Communications Surveys & Tutorials*, 19(3). <https://doi.org/10.1109/COMST.2017.2682318>



Lin, L., Liao, X., Jin, H., Li, P. (2019). "Computation Offloading Toward Edge Computing." *Proceedings of the IEEE*, 107(8). <https://doi.org/10.1109/JPROC.2019.2922285>



Tran, T.X., Hajisami, A., Pandey, P., Pompili, D. (2017). "Collaborative Mobile Clouds : An Architecture for Resource-Limited Mobile Devices." *IEEE Communications Magazine*, 55(7). <https://doi.org/10.1109/MCOM.2017.1600343>



Chen, X., Jiao, L., Li, W., Fu, X. (2016). "Efficient Multi-User Computation Offloading for Mobile-Edge Cloud Computing." *IEEE/ACM Trans. on Networking*, 24(5). <https://doi.org/10.1109/TNET.2015.2487344>



Kumar, K., Liu, J., Lu, Y.H., Bhargava, B. (2013). "A Survey of Computation Offloading for Mobile Systems." *Mobile Networks and Applications*, 18(1). <https://doi.org/10.1007/s11036-012-0368-0>

Références (2/2)



Hunt, P.B., Robertson, D.I., Bretherton, R.D., Royle, M.C. (1982). "The SCOOT On-Line Traffic Signal Optimisation Technique." *Traffic Engineering & Control*, 23(4). TRRL Report LR 1014.



Robertson, D.I., Bretherton, R.D. (1991). "Optimizing Networks of Traffic Signals in Real Time – The SCOOT Method." *IEEE Trans. on Vehicular Technology*, 40(1). <https://doi.org/10.1109/25.69966>



Yousefpour, A., Fung, C., Nguyen, T. et al. (2019). "All One Needs to Know about Fog Computing : A Complete Survey." *Journal of Systems Architecture*, 98. <https://doi.org/10.1016/j.sysarc.2019.02.009>



Deng, R., Lu, R., Lai, C., Luan, T.H., Liang, H. (2016). "Optimal Workload Allocation in Fog-Cloud Computing." *IEEE Communications Letters*, 20(6). <https://doi.org/10.1109/LCOMM.2016.2558185>



Gupta, H., Vahid Dastjerdi, A., Ghosh, S., Buyya, R. (2017). "iFogSim : A Toolkit for Modeling and Simulation of Resource Management in IoT, Edge and Fog Computing." *Software : Practice and Experience*, 47(9). <https://doi.org/10.1002/spe.2509>



Lopez, P.A., Behrisch, M., Bieker-Walz, L. et al. (2018). "Microscopic Traffic Simulation using SUMO." *IEEE Intelligent Transportation Systems Conf. (ITSC)*. <https://doi.org/10.1109/ITSC.2018.8569938>



Al-Sultan, S., Al-Doori, M.M., Al-Bayatti, A.H., Zedan, H. (2014). "A Comprehensive Survey on Vehicular Ad Hoc Network." *Journal of Network and Computer Applications*, 37. <https://doi.org/10.1016/j.jnca.2013.02.036>



Kenney, J.B. (2011). "Dedicated Short-Range Communications (DSRC) Standards in the United States." *Proceedings of the IEEE*, 99(7). <https://doi.org/10.1109/JPROC.2011.2132790>



Miettinen, A.P., Nurminen, J.K. (2010). "Energy Efficiency of Mobile Clients in Cloud Computing." *USENIX HotCloud*. <https://www.usenix.org/conference/hotcloud-10/energy-efficiency-mobile-clients-cloud-computing>



Webster, F.V. (1958). "Traffic Signal Settings." *Road Research Technical Paper No. 39*, HMSO, London. – Fondements de la théorie des files d'attente aux intersections.

Merci pour votre attention

Questions ?